Bài 2. Mô hình ngôn ngữ

Giảng viên: ThS. Võ Ngọc Bích Uyên

Nội dung

- Giới thiệu bài toán dự báo từ và sửa lỗi chính tả
- Định nghĩa mô hình ngôn ngữ
- Mô hình ngôn ngữ dựa trên N-gram
- Đánh giá mô hình
- Zero problem và các phương pháp làm trơn
- Giới thiệu tóm tắt các mô hình dựa trên mạng Neural
- Thực hành: Ứng dụng mô hình ngôn ngữ cho sửa lỗi chính tả

Bài toán dự báo từ và sửa lỗi chính tả

- Dự báo từ: Gợi ý từ/cụm từ tiếp theo trong quá trình soạn thảo.
  - Ví dụ: Soạn tin nhắn trên điện thoại di động
- Sửa lỗi chính tả: Tìm từ có thể sai chính tả và gợi ý từ thay thế cho đúng
- Ứng dụng:
  - Soạn thảo văn bản nhanh và chính xác
    - Người khuyết tật
    - Người không quen thao tác với các thiết bị điện tử
    - Người không thành thạo ngôn ngữ, tránh lỗi ngữ pháp
  - Tăng giá trị của sản phẩm soạn thảo văn bản

Mô hình ngôn ngữ

Mô hình ngôn ngữ

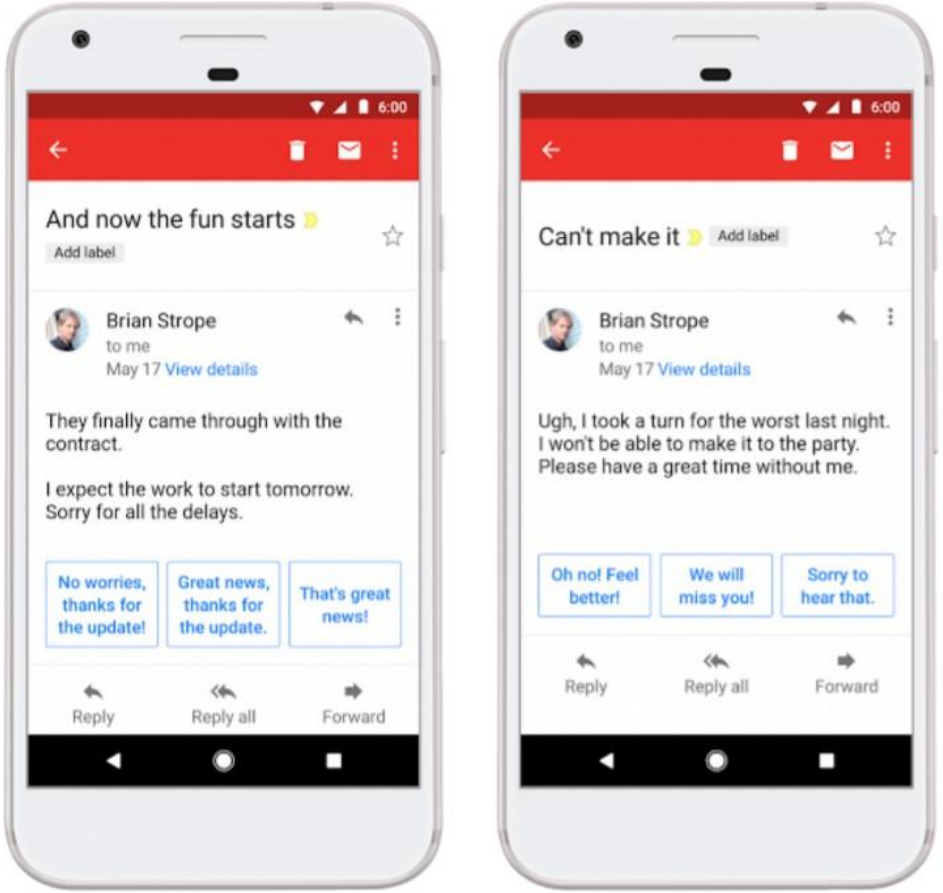
- Xác suất xảy ra câu nói “hôm nay trời đẹp” so với xác suất xảy ra câu nói “trời đẹp nay hôm”?

$$P(\text{“hôm nay trời đẹp”})\ ?\ P(\text{“trời đẹp nay hôm”})$$

Mô hình ngôn ngữ

- Mô hình ngôn ngữ nhằm xác định khả năng (likelihood) một câu (hoặc một chuỗi từ) thuộc một ngôn ngữ.
- Mô hình ngôn ngữ (xác suất) là mô hình cho phép tính xác suất của một câu (chuỗi các từ) trong một ngôn ngữ
- Mô hình ngôn ngữ tốt sẽ đánh giá cao các câu đúng ngữ pháp, trôi chảy hơn các câu có thứ tự từ ngẫu nhiên.
    - vd: P(“hôm nay trời đẹp”) > P(“trời đẹp nay hôm”)

Ứng dụng của mô hình ngôn ngữ



Mô hình ngôn ngữ

- Mục tiêu: gán xác suất cho một câu.
- Machine Translation:
  - P(**high** winds tonight) > P(**large** winds tonight)
- Spell Correction:
  - The office is about fifteen **minuets** from my house
  - P(about fifteen **minutes** from) > P(about fifteen **minuets** from)
- Speech Recognition:
  - P(I saw a van) >> P(eyes awe of an)
- Summarization, question-answering, etc., etc.!!

Mô hình ngôn ngữ

- Mục tiêu: Tính xác suất của một câu hoặc một chuỗi từ có thứ tự:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5, \dots, w_m)$$

- Theo định nghĩa xác suất có điều kiện:

$$P(AB) = P(B|A) \times P(A)$$

- Ta có:

$$P(w_1, w_2, w_3, w_4, w_5, \dots, w_m) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1 w_2) \times \cdots \times P(w_m|w_1 w_2 w_3 \dots w_{m-1})$$

(Chain Rule)

$$P(\text{“hôm nay trời đẹp”}) = P(\text{hôm}) * P(\text{nay}|\text{hôm}) * P(\text{trời}|\text{hôm nay}) * P(\text{đẹp}|\text{hôm nay trời})$$

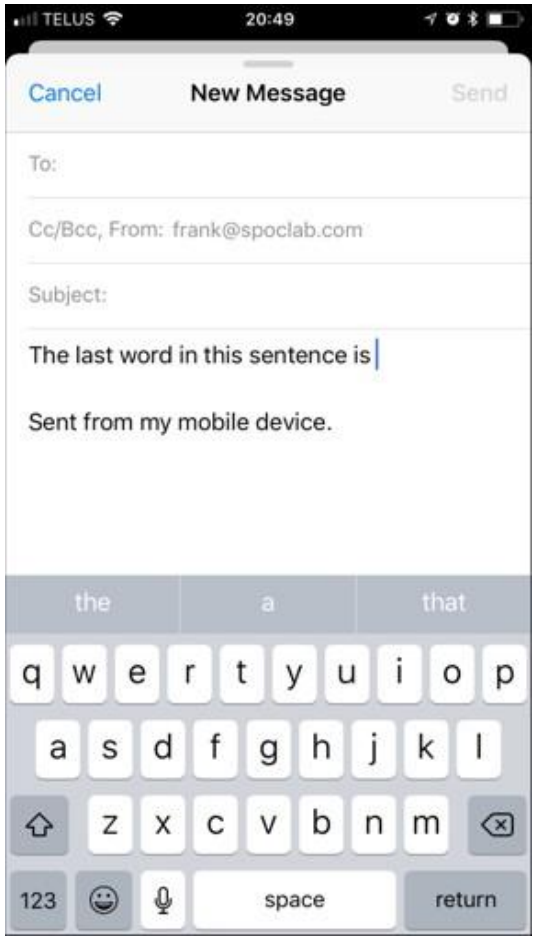
Mô hình ngôn ngữ

- Mục tiêu: Tính xác suất của một từ sắp xuất hiện

$$P(w_m | w_1 w_2 w_3 \ \dots w_{m-1})$$

- Một mô hình tính toán một trong hai xác suất sau:

$P(W)$ hoặc $P(w_m | w_1 w_2 w_3 \ \dots w_{m-1})$ được gọi là mô hình ngôn ngữ.



Mô hình ngôn ngữ N-gram

- Cách tính xác suất:

$$P(\text{đẹp}|\text{hôm nay trời}) = \frac{P(\text{hôm nay trời đẹp})}{P(\text{hôm nay trời})}$$

- Không thể đếm và lưu hết các xác suất trên, đặc biệt với m là độ dài văn bản ngôn ngữ tự nhiên
  - sử dụng chuỗi Markov bậc n với giả thiết 1 từ chỉ phụ thuộc n từ đứng trước nó (mô hình n-gram)
  - $P(w_m|w_1w_2w_3 \ldots w_{m-1}) = P(w_m|w_{m-n}w_{m-n+1}w_{m-n+2} \ldots w_{m-1})$

Giả thuyết Markov

- Mô hình Markov là một lớp mô hình xác suất, giả định rằng xác suất của một đơn vị trong tương lai chỉ phụ thuộc vào một số hữu hạn đơn vị gần nhất trong quá khứ mà không cần xét toàn bộ lịch sử trước đó.
    - Mô hình bigram được gọi là mô hình Markov bậc nhất (vì nó nhìn vào một token trong quá khứ);
    - Một trigram được gọi là mô hình Markov bậc hai;
    - Nói chung, một N-Gram được gọi là mô hình Markov bậc N-1.

Các mô hình N-gram

- Mô hình unigram: $P(w_1 w_2 \dots w_n) \approx \prod_{i=1}^{n} P(w_i)$

  $P(\text{“hôm nay trời đẹp”}) = P(\text{hôm}) * P(\text{nay}) * P(\text{trời}) * P(\text{đẹp})$

- Mô hình bigram: $P(w_1 w_2 \dots w_n) \approx P(w_1) \prod_{i=2}^{n} P(w_i | w_{i-1})$

  $P(\text{“hôm nay trời đẹp”}) = P(\text{hôm}) * P(\text{nay}|\text{hôm}) * P(\text{trời}|\text{nay}) * P(\text{đẹp}|\text{trời})$

- Mô hình trigram: $P(w_1 w_2 \dots w_n) \approx P(w_1) P(w_2 | w_1) \prod_{i=3}^{n} P(w_i | w_{i-1}, w_{i-2})$

  $P(\text{“hôm nay trời đẹp”}) = P(\text{hôm}) * P(\text{nay}|\text{hôm}) * P(\text{trời}|\text{hôm nay}) * P(\text{đẹp}|\text{nay trời})$

Ước lượng xác suất N-gram

- Việc ước lượng xác suất N-gram được gọi là Ước lượng hợp lý cực đại (Maximum Likelihood Estimation – MLE).
- Ta thu được ước lượng MLE cho các tham số của mô hình N-gram bằng cách:
  - Đếm số lần xuất hiện (count) của các N-gram trong corpus;
  - Chuẩn hóa các giá trị đếm để xác suất nằm trong khoảng [0,1].
- Ước lượng xác suất Bigram:

$$P(w_i|w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

Trong đó:

$count(w_{i-1}, w_i)$ số lần bigram (w_{i-1}, w_i) xuất hiện trong corpus;

$count(w_{i-1})$ số lần từ w_{i-1}xuất hiện trong corpus

- Ước lượng xác suất N-gram (tổng quát)

$$P\left(w_i|w_{i-n+1}^{i-1}\right) = \frac{count\left(w_{i-n+1}^{i-1}, w_i\right)}{count\left(w_{i-n+1}^{i-1}\right)}$$

Ước lượng xác suất N-gram

- Corpus: Ta bổ sung ký hiệu đặc biệt <s> ở đầu mỗi câu → nhằm cung cấp ngữ cảnh bigram cho từ đầu tiên; đồng thời thêm ký hiệu kết thúc câu </s>
    - <s> I am Sam </s>
    - <s> Sam I am </s>
    - <s> I fly </s>
- Từ vựng (unique words): Các từ xuất hiện trong corpus: I, am, Sam, fly
- Unigram

$$P(\mathrm{I}) = \frac{3}{8};\ P(\mathrm{am}) = \frac{2}{8};\ P(\mathrm{Sam}) = \frac{2}{8};\ \ P(\mathrm{fly}) = \frac{1}{8}$$

Ước lượng xác suất bigram

- Bigram
  - <s> và </s> cũng được xem là token
  - Tổng số token: 4 từ + <s> + </s> = 6 token
  - Tổng số bigram có thể có: $6 \times 6 = 36$ bigram

<s> I am Sam </s>

<s> Sam I am </s>

<s> I fly </s>

	I	Sam	am	fly	<s>	</s>
<s>	P(I|<s>)=2/3	P(Sam|<s>)=1/3	P(am|<s>)=0	P(fly|<s>)=0	P(<s>|<s>)=0	P(</s>|<s>)=0
I	P(I|I)=0	P(Sam|I)=0	P(am|I)=2/3	P(fly|I)=1/3	P(<s>|I)=0	P(</s>|I)=0
am	P(I|am)=0	P(Sam|am)=1/2	P(am|am)=0	P(fly|am)=0	P(<s>|am)=0	P(</s>|am)=1/2
Sam	P(I|Sam)=1/2	P(Sam|Sam)=0	P(am|Sam)=0	P(fly|Sam)=0	P(<s>|Sam)=0	P(</s>|Sam)=1/2
fly	P(I|fly)=0	P(Sam|fly)=0	P(am|fly)=0	P(fly|fly)=0	P(<s>|fly)=0	P(</s>|fly)=1
</s>	P(I|</s>)=0	P(Sam|</s>)=0	P(am|</s>)=0	P(fly|</s>)=0	P(<s>|</s>)=1 or ?	P(</s>|</s>)=0

Ước lượng xác suất trigram

- Trigram
  - Có $6 \times 6 \times 6 = 216$ trigram.
  - Giả sử có hai ký tự <s> <s> ở đầu và hai ký tự </s> </s> ở cuối.

$P(\mathrm{I}|\text{<s><s>}) = \frac{2}{3}$

$P(\mathrm{am}|\text{<s>I}) = \frac{1}{2}$

$P(\mathrm{Sam}|\mathrm{I\ am}) = \frac{1}{2}$

$P(\text{</s>}|\mathrm{I\ am}) = \frac{1}{2}$

$P(\mathrm{Sam}|\text{<s><s>}) = \frac{1}{3}$

$P(\mathrm{fly}|\text{<s>I}) = \frac{1}{2}$

$P(\mathrm{I}|\text{<s>Sam}) = 1$

$P(\text{</s>}|\mathrm{am\ Sam}) = 1$

$P(\text{</s>}|\mathrm{Sam}\ \text{</s>}) = 1$

<s> I am Sam </s>

<s> Sam I am </s>

<s> I fly </s>

Bài tập: Ước lượng xác suất bigram

- Corpus: *Berkeley Restaurant Project Sentences*
  - Corpus gồm 9.332 câu.
  - Bảng thống kê thể hiện số lần xuất hiện (raw counts) của các bigram tương ứng với 8 từ được chọn (trong tổng số 1.446 từ trong corpus)

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Bài tập: Ước lượng xác suất bigram

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

- Unigram count:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Chuẩn hóa bigram theo unigram count: Giá trị xác suất của bigram

Bài tập: Ước lượng xác suất bigram

- Chuẩn hóa bigram theo unigram count: Giá trị xác suất của bigram

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Bài tập: Ước lượng xác suất câu bằng bigram

- Một số cặp bigram khác:

  $P(\mathrm{I}|\text{<s>}) = 0.25$

  $P(\mathrm{english}|\mathrm{want}) = 0.0011$

  $P(\mathrm{food}|\mathrm{english}) = 0.5$

  $P(\text{</s>}|\mathrm{food}) = 0.68$

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

- Tính xác suất của câu “I want English food"

Bài tập: Ước lượng xác suất câu bằng bigram

- Một số cặp bigram khác:

  $P(\text{I}|\text{<s>}) = 0.25$

  $P(\text{english}|\text{want}) = 0.0011$

  $P(\text{food}|\text{english}) = 0.5$

  $P(\text{</s>}|\text{food}) = 0.68$

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

- Xác suất của câu “I want English food"

$$P(\text{<s> I want english food</s>})$$
$$= P(\text{I}|\text{<s>})\, P(\text{want}|\text{I})\, P(\text{english}|\text{want})\, P(\text{food}|\text{english})\, P(\text{</s>}|\text{food})$$
$$= 0.25 \times 0.33 \times 0.0011 \times 0.5 \times 0.68 = 0.000031$$

Bài tập: Ước lượng xác suất câu

Cho tập dữ liệu văn bản gồm các câu sau:

<s> cô ấy dạy môn tin học </s>

<s> anh dạy môn toán </s>

<s> cô ấy học toán anh ấy dạy </s>

<s> môn toán môn tin đều hay </s>

<s> anh ấy dạy môn toán hay môn tin </s>

Xây dựng mô hình ngôn ngữ unigram và bigram?

Triển khai thực tế – Log Probabilities

- Xác suất luôn nhỏ hơn hoặc bằng 1, nên:
  - Khi nhân nhiều xác suất với nhau, kết quả sẽ rất nhỏ
  - Nhân đủ nhiều N-gram có thể dẫn đến tràn số dưới (numerical underflow)

⇒ Sử dụng log-probabilities: thay vì dùng xác suất gốc, ta sử dụng log của xác suất

- Các giá trị log không quá nhỏ (Tránh được kết quả về 0)
- Cộng trong không gian log tương đương với nhân trong không gian tuyến tính (Cộng nhanh hơn nhân)

$$log(p_1 \times p_2 \times p_3) = logp_1 + logp_2 + logp_3$$

Đánh giá mô hình ngôn ngữ

Đánh giá các mô hình ngôn ngữ

- Mô hình ngôn ngữ có ưu tiên câu “tốt” hơn câu “xấu” không?

  Mô hình ngôn ngữ có gán xác suất cao hơn cho: *các câu tự nhiên; các câu xuất hiện thường xuyên trong dữ liệu* so với: *các câu không đúng ngữ pháp; các câu hiếm gặp hoặc không tự nhiên* hay không?
- Huấn luyện và đánh giá mô hình
  - Huấn luyện các tham số của mô hình trên tập huấn luyện (training set).
  - Đánh giá hiệu năng của mô hình trên dữ liệu chưa từng thấy - tập kiểm tra (test set)
  - Test set là:
    - Một tập dữ liệu chưa từng xuất hiện trong quá trình huấn luyện
    - Hoàn toàn độc lập với training set
- Đánh giá mô hình: Chỉ số đánh giá (evaluation metric) cho biết mô hình hoạt động tốt đến mức nào trên test set

Đánh giá ngoài (Extrinsic evaluation)

- Đánh giá ngoài của mô hình ngôn ngữ N-gram là sử dụng mô hình trong một tác vụ cụ thể và đo lường mức độ cải thiện của tác vụ đó.
- So sánh 2 mô hình ngôn ngữ A và B
  - Sử dụng mỗi mô hình cho cùng một tác vụ: sửa lỗi chính tả, nhận dạng tiếng nói, dịch máy,…
  - Thử nghiệm (chạy) nhiệm vụ đó, tính độ chính xác khi sử dụng mô hình A và B
    - Bao nhiêu từ sai được sửa đúng?
    - Bao nhiêu từ được dịch đúng?
  - So sánh độ chính xác khi sử dụng A và B
    - Mô hình nào cho độ chính xác cao hơn → mô hình tốt hơn
- Nhược điểm: Đánh giá ngoài thường tốn thời gian và tài nguyên

Đánh giá trong (Intrinsic evaluation)

- Đánh giá trong là đánh giá chất lượng mô hình độc lập với bất kỳ tác vụ cụ thể nào.
- Khi có một corpus văn bản và cần so sánh hai mô hình N-gram khác nhau:
  - Chia dữ liệu thành: Tập huấn luyện (training set) và tập kiểm tra (test set)
  - Huấn luyện cả hai mô hình trên training set
  - So sánh mức độ phù hợp của hai mô hình trên test set
- Tiêu chí so sánh: Mô hình nào gán xác suất cao hơn cho tập test → mô hình tốt hơn
- Trong thực tế:
  - Không dùng xác suất thô mà sử dụng chỉ số dựa trên xác suất gọi là Perplexity
  - Perplexity là thước đo chuẩn để đánh giá mô hình ngôn ngữ

Độ phức tạp (perplexity)

- Mô hình ngôn ngữ tốt nhất là mô hình dự đoán từ chưa nhìn thấy tốt nhất trên tập kiểm tra
  - Cho xác suất câu cao nhất P(sentence)
  - Perplexity là nghịch đảo xác suất trên tập test, chuẩn hóa theo số từ
- Tập test: W = $w_1 w_2 \dots w_N$
- Perplexity: $PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \dots w_{i-1})}}$ (Chain rule)
- Với bigram: $PP(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1})}}$
- Giảm thiểu perplexity cũng đồng nghĩa với việc tối đa hóa xác suất.

Độ phức tạp (perplexity)

- Perplexity phản ánh số lượng lựa chọn trung bình (các nhánh có thể xảy ra) cho từ tiếp theo trong ngôn ngữ, tức là mức độ phức tạp của mô hình ngôn ngữ.
- Giả thiết 1 câu gồm các chữ số ngẫu nhiên. Khi đó độ phức tạp của câu dựa trên 1 mô hình sẽ gán PP=10

$$PP(W) = P(w_1 w_2 \dots w_w)^{-\frac{1}{N}} = \left(\frac{1}{10}^{N}\right)^{-\frac{1}{N}} = \frac{1}{10}^{-1} = 10$$

Lower perplexity = better model

- Corpus: Wall Street Journal (WSJ)
- Training 38 million words, test 1.5 million words

N-gram Order			
Perplexity	962	170	109

Lower perplexity = better model

- Corpus: Wall Street Journal (WSJ)
- Training 38 million words, test 1.5 million words

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

Unknown Words

- Trong thực tế, ta luôn phải xử lý những từ chưa từng xuất hiện trong dữ liệu huấn luyện, gọi là từ không biết (unknown words).
- Để mô hình hóa các từ này trong tập test, ta thêm một từ giả (pseudo-word) ký hiệu là <UNK> vào tập huấn luyện.
- Cách xử lý từ không biết:
  - Thay thế các từ hiếm trong dữ liệu huấn luyện bằng <UNK>, dựa trên tần suất xuất hiện.

    Ví dụ:

    - Thay thế tất cả các từ xuất hiện ít hơn n lần trong tập huấn luyện bằng <UNK> (với *n* là một số nhỏ), hoặc
    - Xác định trước kích thước từ vựng V (ví dụ 50.000 từ), chọn V từ có tần suất cao nhất. Thay thế tất cả các từ còn lại bằng <UNK>
    - Sau đó, huấn luyện mô hình ngôn ngữ như bình thường, coi <UNK> là một từ thông thường.

Hiện tượng quá khớp dữ liệu (overfitting)

- Mô hình N-gram, giống như nhiều mô hình thống kê khác, phụ thuộc mạnh vào corpus huấn luyện. Các xác suất học được thường mã hóa những đặc điểm riêng của corpus đó.
- N-gram chỉ hoạt động tốt khi corpus kiểm tra (test) có phân bố tương tự corpus huấn luyện (training).
- Trong thực tế điều này thường không đúng. Có những sự kiện:
  - Không xuất hiện trong tập huấn luyện
  - Nhưng xuất hiện trong tập kiểm tra

  → Điều này dẫn đến xác suất bằng 0 (zeros probability problem).

→ Ta cần tạo ra các mô hình đủ mạnh để tổng quát hóa: Loại bỏ xác suất 0

Trường hợp xác suất = 0

Tập train:

- … denied the allegations
- … denied the reports
- … denied the claims
- … denied the request

Tập test

- … denied the offer
- … denied the loan

$P(\text{offer} \mid \text{denied the}) = 0$ → xác suất của 1 câu hoặc một cụm từ bằng 0

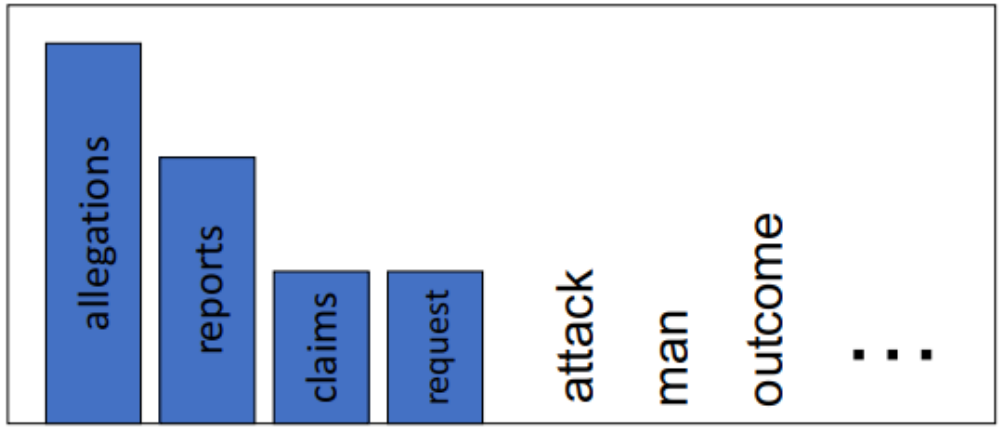
→ Sử dụng các phương pháp làm mịn

Ý tưởng của phương pháp làm mịn

Xác suất trên tập train:

- $P(\mathrm{w} \mid \text{denied the})$
  - 3 allegations
  - 2 reports
  - 1 claims
  - 1 request

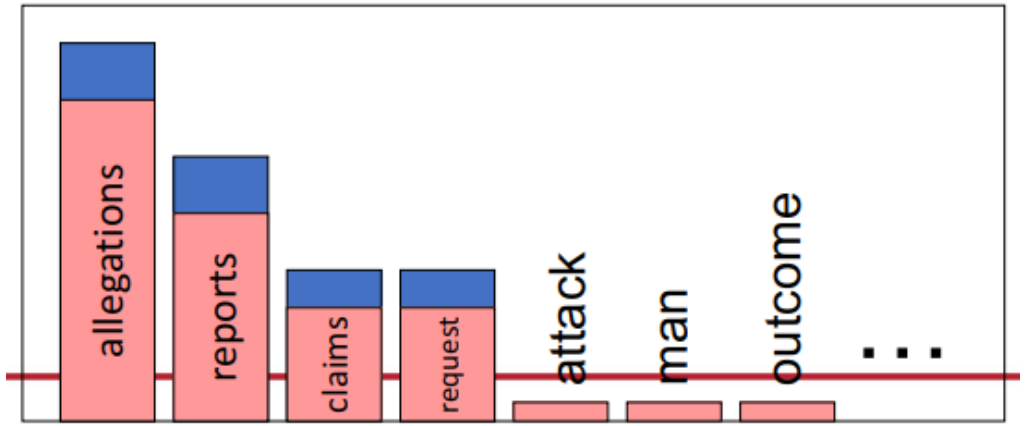
  = 7 total



Giảm xác suất các n-gram có xác suất lớn hơn 0 để bù cho các n-gram có xác suất bằng 0.

- $P(\mathrm{w} \mid \text{denied the})$
  - 2.5 allegations
  - 1.5 reports
  - 0.5 claims
  - 0.5 request
  - 2 other

  = 7 total



Laplace (Add-one) smoothing

- Giả thiết mỗi từ xuất hiện nhiều hơn 1 lần so với thực tế
- Cộng 1 vào tất cả các giá trị đếm

$$P_{MLE}(w_i|w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

$$P_{Laplace}(w_i|w_{i-1}) = \frac{count(w_{i-1}, w_i) + 1}{count(w_{i-1}) + V}$$

Laplace-smoothed bigrams

Corpus: *Berkeley Restaurant Project Sentences*

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Laplace-smoothed bigrams

Corpus: *Berkeley Restaurant Project Sentences*

$$P_{Laplace}(w_i|w_{i-1}) = \frac{count(w_{i-1}, w_i) + 1}{count(w_{i-1}) + V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Add-k smoothing

- Add-one smoothing gây ra sự thay đổi rất lớn trong các giá trị đếm:

  VD: $P(\text{to} \mid \text{want})$ giảm từ 0.66 (không smoothing) xuống còn 0.26 (có smoothing)

- Sự thay đổi lớn trong counts và probabilities xảy ra vì quá nhiều khối xác suất bị phân bổ cho các sự kiện chưa từng xuất hiện.

→ Phương pháp Add-k Smoothing

  - Chỉ chuyển một lượng nhỏ xác suất từ các sự kiện đã thấy sang các sự kiện chưa thấy.
  - Thay vì cộng 1 vào mỗi count ta cộng một giá trị phân số k (ví dụ: 0.5, 0.05, 0.01)

$$P_{add_k}(w_i|w_{i-1}) = \frac{count(w_{i-1}, w_i) + k}{count(w_{i-1}) + kV}$$

Smoothing: Backoff

Đôi khi sử dụng ít ngữ cảnh hơn lại hiệu quả

Trong backoff:

- Ta sử dụng trigram nếu có đủ dữ liệu
- Nếu không, lùi xuống bigram
- Nếu vẫn không đủ, lùi tiếp xuống unigram

Ta chỉ “back off” sang N-gram bậc thấp hơn khi N-gram bậc cao hơn có TH xác suất bằng 0 (zero counts).

Smoothing: Interpolation (Nội suy)

Interpolation: Kết hợp các ước lượng xác suất từ trigram, bigram và unigram bằng cách gán trọng số và cộng chúng lại.

Simple linear interpolation:

$$P_{SLI}(w_n|w_{n-1}, w_{n-2}) = \lambda_1 P(w_n|w_{n-1}, w_{n-2}) + \lambda_2 P(w_n|w_{n-1}) + \lambda_3 P(w_n)$$

Trong đó $\sum_i \lambda_i = 1$

Interpolation with context-conditioned weights:

$$P_{ICCW}(w_n|w_{n-1}, w_{n-2})$$

$$= \lambda_1\left(w_{n-2}^{n-1}\right)P(w_n|w_{n-1}, w_{n-2}) + \lambda_2\left(w_{n-2}^{n-1}\right)P(w_n|w_{n-1}) + \lambda_3\left(w_{n-2}^{n-1}\right)P(w_n)$$

Trong phiên bản phức tạp hơn, mỗi trọng số λ được tính có điều kiện theo ngữ cảnh

Interpolation với trọng số phụ thuộc ngữ cảnh

- Sử dụng một held-out corpus

- Chọn các trọng số λ sao cho tối đa hóa xác suất của dữ liệu held-out
- Quy trình:
  - Cố định các xác suất N-gram (học từ training set)
  - Tìm các giá trị λ sao cho xác suất trên held-out set là cao nhất
  - Có nhiều phương pháp khác nhau để tìm bộ λ tối ưu

Các mô hình N-gram

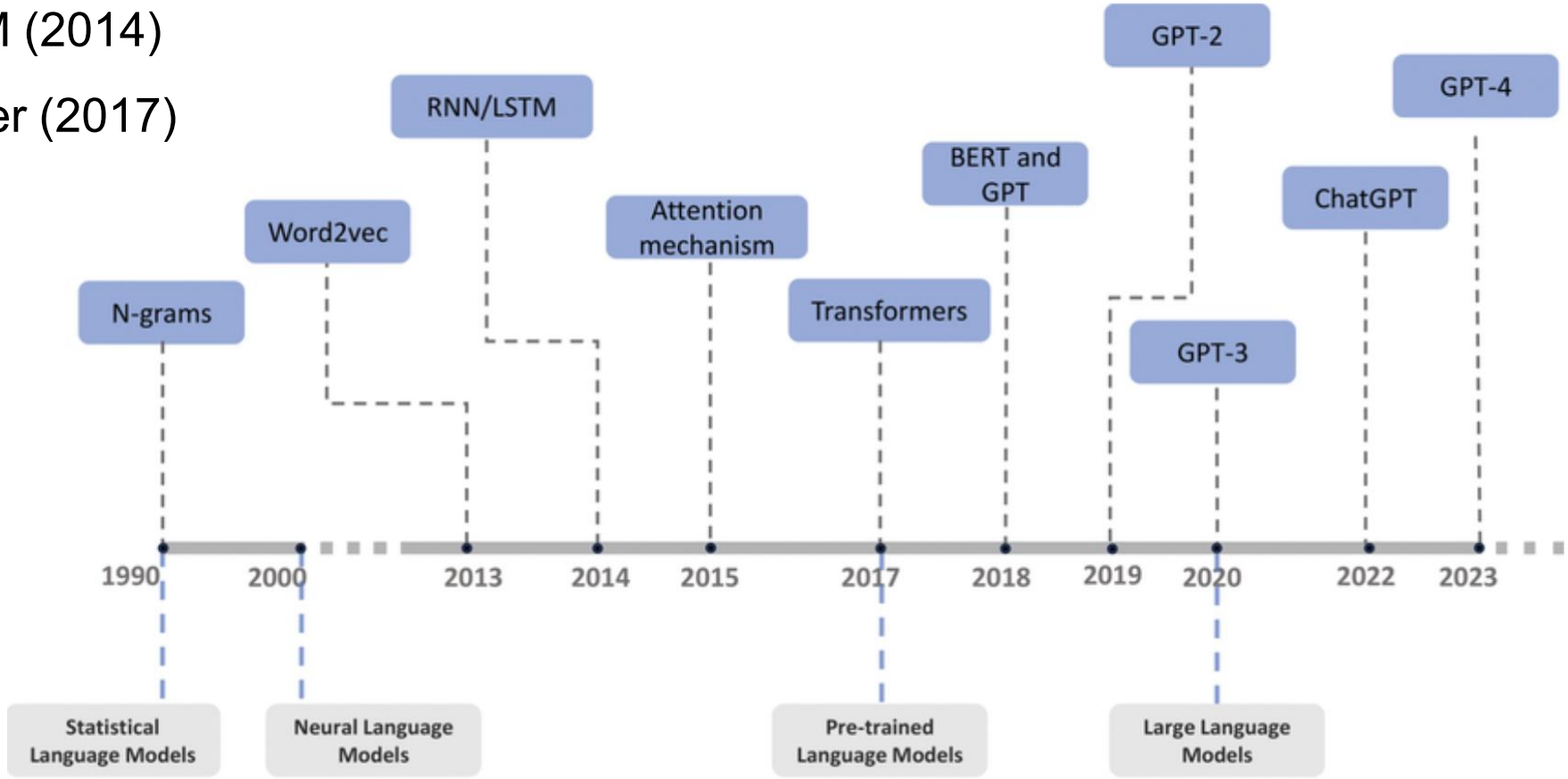
- Nên sử dụng N-Gram nào làm mô hình ngôn ngữ?
  - N càng lớn → mô hình càng chính xác
    - Nhưng việc ước lượng xác suất N-gram sẽ kém tin cậy
    - Bảng N-gram trở nên rất thưa (sparse)
  - N nhỏ → mô hình kém chính xác hơn
    - Nhưng ước lượng xác suất tốt hơn
    - Bảng N-gram ít thưa hơn
- Kích thước bảng N-Gram (với 10.000 từ vựng)
  - Unigram: $10{,}000$
  - Bigram: $10{,}000 \times 10{,}000 = 100{,}000{,}000$
  - Trigram: $10{,}000 \times 10{,}000 \times 10{,}000 = 1{,}000{,}000{,}000{,}000$
- Trong thực tế
  - Thông thường không sử dụng N-gram lớn hơn Trigram
  - Thậm chí trong nhiều hệ thống, Bigram đã là đủ

Hạn chế của mô hình ngôn ngữ N-gram

- Nhiều khi, ngữ cảnh cục bộ (local context) không cung cấp đủ thông tin hữu ích để dự đoán, mà các phụ thuộc xa (long-distance dependencies) mới là nguồn manh mối quan trọng.
- Phụ thuộc cú pháp
  - *The man next to the large oak tree near the grocery store on the corner is tall.*
  - *The men next to the large oak tree near the grocery store on the corner are tall.*
- Phụ thuộc ngữ nghĩa
  - *The bird next to the large oak tree near the grocery store on the corner flies rapidly.*
  - *The man next to the large oak tree near the grocery store on the corner talks rapidly.*

Neural Network Language Models

- Neural Language Model (2000)
- RNN/LSTM (2014)
- Transformer (2017)



Biểu diễn từ bằng vector

- Xây dựng một mô hình nghĩa của từ tập trung vào sự tương đồng
- Mỗi từ = một vector
- Các từ tương tự ở gần nhau trong không gian

Biểu diễn từ bằng vector

- Các mô hình dựa trên neuron network sử dụng Word2Vec có tính tổng quát cao.

Dimensions

Word vectors				
dog	-0.4	0.37	0.02	-0.34
cat	-0.15	-0.02	-0.23	-0.23
lion	0.19	-0.4	0.35	-0.48
tiger	-0.08	0.31	0.56	0.07
elephant	-0.04	-0.09	0.11	-0.06
cheetah	0.27	-0.28	-0.2	-0.43
monkey	-0.02	-0.67	-0.21	-0.48
rabbit	-0.04	-0.3	-0.18	-0.47
mouse	0.09	-0.46	-0.35	-0.24
rat	0.21	-0.48	-0.56	-0.37

animal
domesticated
pet
fluffy

- Khi dùng từ (words): Yêu cầu từ phải xuất hiện giống hệt trong cả tập huấn luyện và tập kiểm tra
- Khi dùng embedding: Vẫn hoạt động tốt nếu các từ tương tự về nghĩa đã xuất hiện trong dữ liệu huấn luyện

Neural Language Models

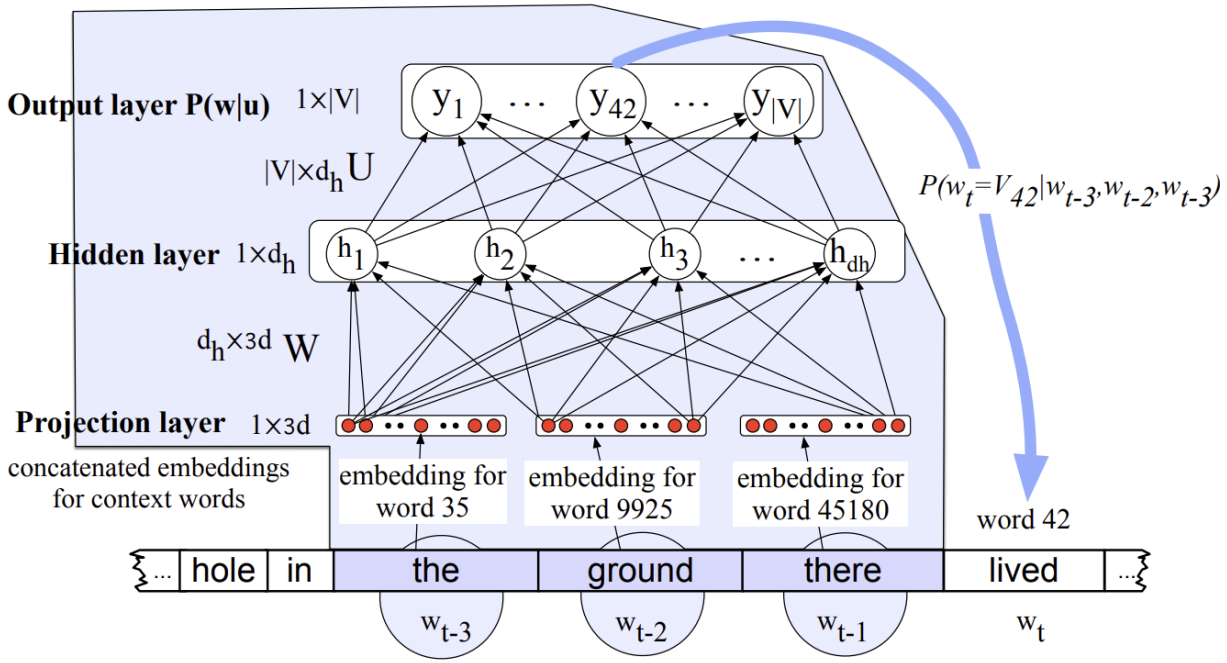
Mô hình ngôn ngữ nơ-ron sử dụng embedding + mạng nơ-ron để dự đoán từ tiếp theo một cách liên tục và tổng quát hơn N-gram.

4) Softmax đoán từ tiếp theo

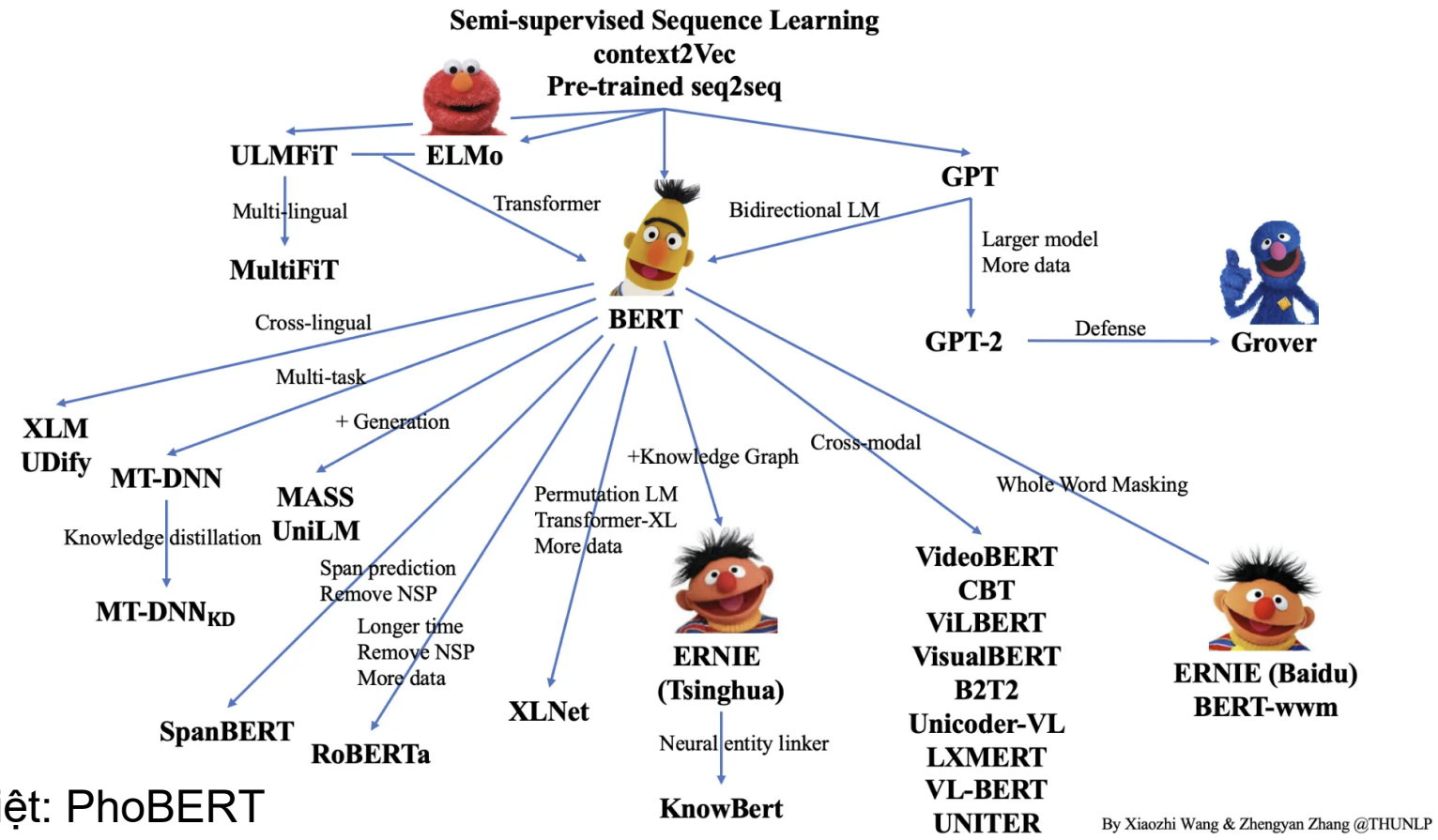
3) Mạng nơ-ron học mẫu

2) Ghép các từ ngữ cảnh

1) Biến từ → vector (embedding)



Các mô hình ngôn ngữ huấn luyện trước



Tiếng Việt: PhoBERT

- Dựa trên RoBERTa huấn luyện theo thủ tục tương tự như BERT.
- Có 2 phiên bản base và large

Thực hành

Ứng dụng mô hình ngôn ngữ cho sửa lỗi chính tả

Thực hành

Mục tiêu: xây dựng chương trình kiểm lỗi cú pháp tiếng Anh đơn giản

Bài 1:

- Xây dựng mô hình ngôn ngữ dựa trên n-gram sử dụng phương pháp smoothing là Laplace, cho các mô hình: 1-gram; 2-gram và 3-gram
- Tính xác suất của một câu và tính Perplexity của một câu dựa theo 1-gram, 2-gram, 3-gram
- Phân tích kết quả

Bài 2:

- Cải tiến mô hình bằng cách sử dụng smoothing là phương pháp interpolation theo “Stupid backoff” (Brants et al. 2007)
- So sánh với kết quả bài 1
- Dùng mô hình vừa xây dựng để sinh các từ tiếp theo với một chuỗi từ cho trước.
- Kết hợp với hàm tính khoảng cách giữa các từ để dự đoán từ đúng cho vị trí từ sai chính tả. (from difflib import get_close_matches)